Zephyr Project

IT-Schulungen.com
Technology & Transformation

Ausführungsmodell des Kernels

Threads, Zustände und Ausführungskontexte

2.02

Was wir in 2.02 klären

Vom Main-Thread bis zum Kontextwechsel

01

main() ist ein Thread

Startpunkt des Ausführungsmodells

02

Bausteine

Kontrollblock, Stack und Entry

03

Zustände

ready, running, wartend und beendet

04

Kontexte

Thread, ISR und Workqueue

05

Kurzcheck

Zustände sicher zuordnen

Leitfrage: Wer darf jetzt laufen – und was geschieht mit den anderen?

main() läuft als Thread

QUELLCODE

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

int main(void)
{
    printk("Zephyr startet\n");

  while (1) {
        k_sleep(K_SECONDS(1));
    }
}
```

01 Start

Der Kernel initialisiert das System und ruft main() im Main-Thread auf. main() besitzt Priorität, Zustand und einen eigenen Stack.

02 Blockieren

k_sleep() versetzt den Main-Thread in den Zustand sleeping. Währenddessen wählt der Scheduler einen anderen bereiten Thread.

main() ist kein Sonderfall außerhalb des Kernels, sondern ein normal verwalteter Thread.

Anatomie eines Threads

MODELL

Thread = Ausführung + Zustand + Ressourcen

`Kontrollblock`	k_thread speichert Status, Priorität und Queue-Verknüpfungen
`Eigener Stack`	enthält lokale Daten, Aufrufrahmen und gesicherten Kontext
`Entry-Funktion`	definiert den ausführbaren Ablauf und seine Parameter
`Priorität und Zustand`	bestimmen Ausführbarkeit und Auswahl durch den Scheduler

Jeder zusätzliche Thread kostet Stack-Speicher, Verwaltungsdaten und Kontextwechselzeit.

Thread-Zustände im Überblick

`ready`	ausführbar und in der Ready Queue eingeordnet
`running`	wird gerade auf einer CPU ausgeführt
`pending / sleeping`	wartet auf ein Ereignis oder bis ein Timeout abläuft
`suspended`	explizit angehalten; erst resume macht den Thread wieder bereit
`terminated`	Entry-Funktion beendet oder Thread abgebrochen

Start, Präemption, Warten, Timeout, Suspend und Ende lösen die Zustandswechsel aus.

Was beim Kontextwechsel passiert

AUSLÖSER

höher priorisierter Thread wird ready

1 · **Kontext sichern**	Register und Stack Pointer des laufenden Threads
2 · **Zustand aktualisieren**	running wird je nach Ursache ready oder wartend
3 · **Ready Queue prüfen**	höchste ausführbare Priorität auswählen
4 · **Kontext laden**	Register und Stack Pointer des nächsten Threads
5 · **Ausführung fortsetzen**	der ausgewählte Thread wechselt zu running

Ein Kontextwechsel kostet Laufzeit: Er sichert und lädt Ausführungszustand.

Drei Ausführungskontexte

01	`Thread`	eigener Stack · darf blockieren · vom Scheduler verwaltet
02	`ISR`	reagiert auf Interrupt · kurz halten · nicht blockieren
03	`Workqueue`	Handler läuft in einem Thread · kurze Arbeit aufschieben

Faustregel: ISR erfasst das Ereignis, Workqueue erledigt kurze Folgearbeit, eigener Thread steuert längere Abläufe.

Kurzcheck: Ausführungsmodell

BEGRIFFE

Sicher zuordnen

main() ist ein Thread

ready ist ausführbar

sleeping wartet auf Zeit

terminated läuft nicht mehr

ENTSCHEIDUNG

Wer läuft als Nächstes?

Ready Queue enthält ausführbare Threads

Priorität steuert die Auswahl

Präemption kann running beenden

Warten macht einen Thread nicht bereit

TRANSFER

Ordne den Kontext zu

Sensor-Interrupt: ISR

kurze Folgearbeit: Workqueue

dauerhafter Ablauf: Thread

Lernziel: aktueller Zustand, auslösendes Ereignis und nächster ausführbarer Thread erklärbar.